



Инфраструктура вычислений в биоинформатике

Лекция 7. Продвинутая контейнеризация.
K8s. Экспонирование контейнера в сеть.
Arptainer. Harbor для хранения образов.

Демоны

Демон – это компьютерная программа на хост-системе, которая выполняется как фоновый процесс, не находится под непосредственным управлением пользователя



Инструменты запуска контейнеров

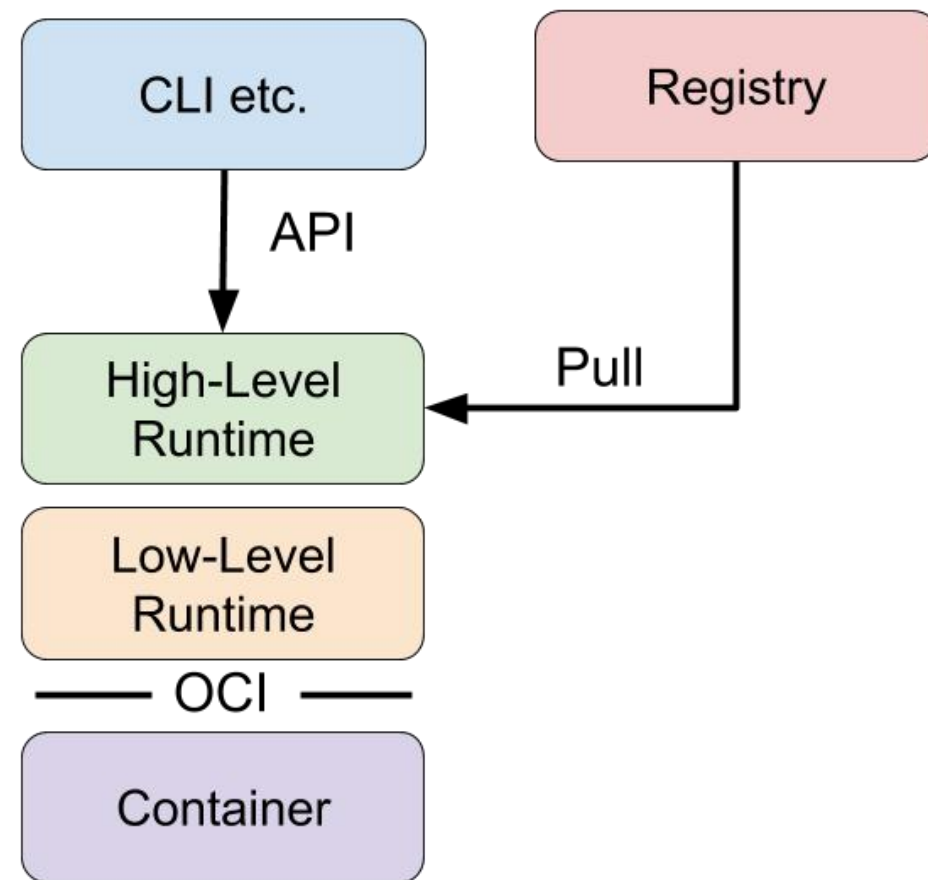
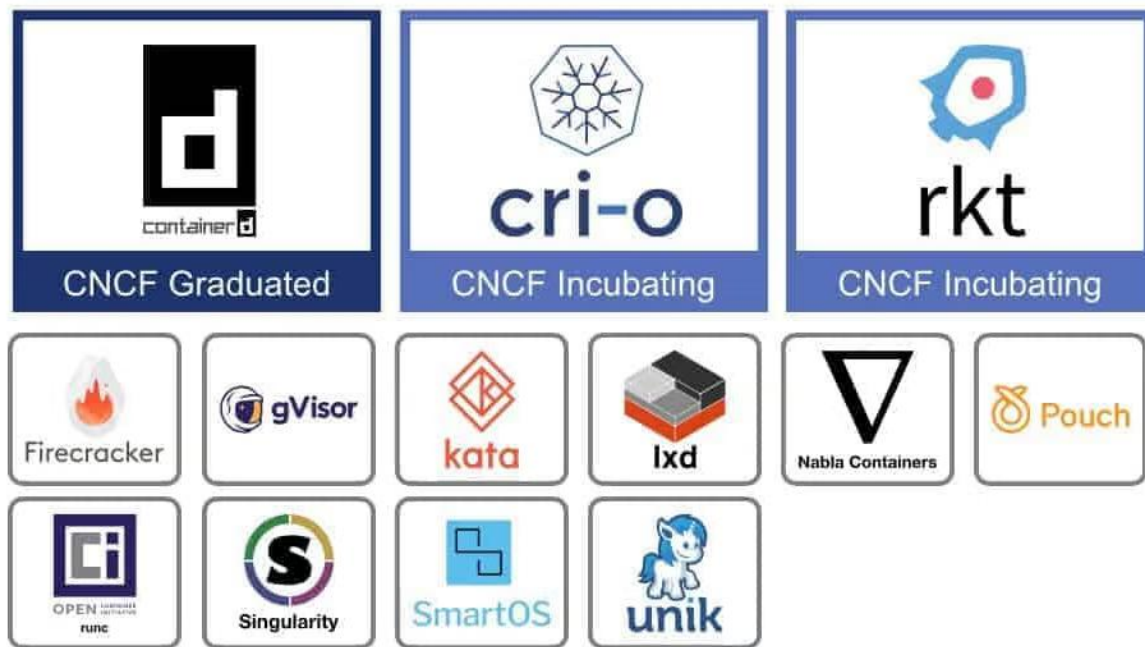
runc – инструмент CLI для создания и запуска контейнеров в соответствии со спецификацией OCI, использует **libcontainer**.

runc подготавливает namespaces и cgroups.

containerd – высокоуровневая среда выполнения контейнеров, демон, который использует **runc** и реализует другие высокоуровневые функции (управление образами).



Runtime контейнера



Демон containerd

Стандарт container runtime, управляет полным жизненным циклом контейнеров.

containerd используется платформами контейнеризации и оркестрации как среда выполнения контейнеров.

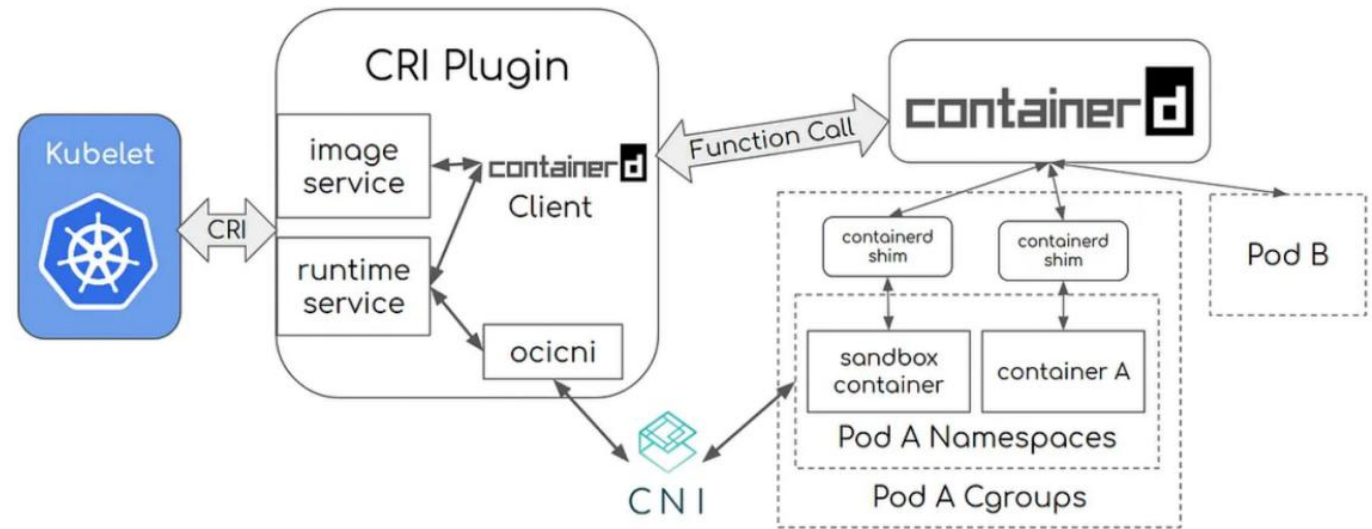
Изначально был в Docker, теперь является независимым проектом и используется по умолчанию в платформе **Kubernetes**.

Kubernetes — оркестратор контейнеров

Основные понятия:

- **Под** — самая маленькая единица в K8s. Обычно один контейнер = один Pod, но иногда в Pod кладут 2–3 тесно связанных контейнера.
- **Deployment** — объект, который требует всегда 3 работающих копии этого Pod. Если один упал, K8s автоматически запустит новый.

Kubernetes — оркестратор контейнеров



Приложение (внутри подов) становится доступным:

- другим сервисам внутри кластера
- внешним пользователям / браузерам / API-клиентам снаружи

Зачем?

Когда есть микросервисное приложение из 15–20 компонентов. Для каждого нужно:

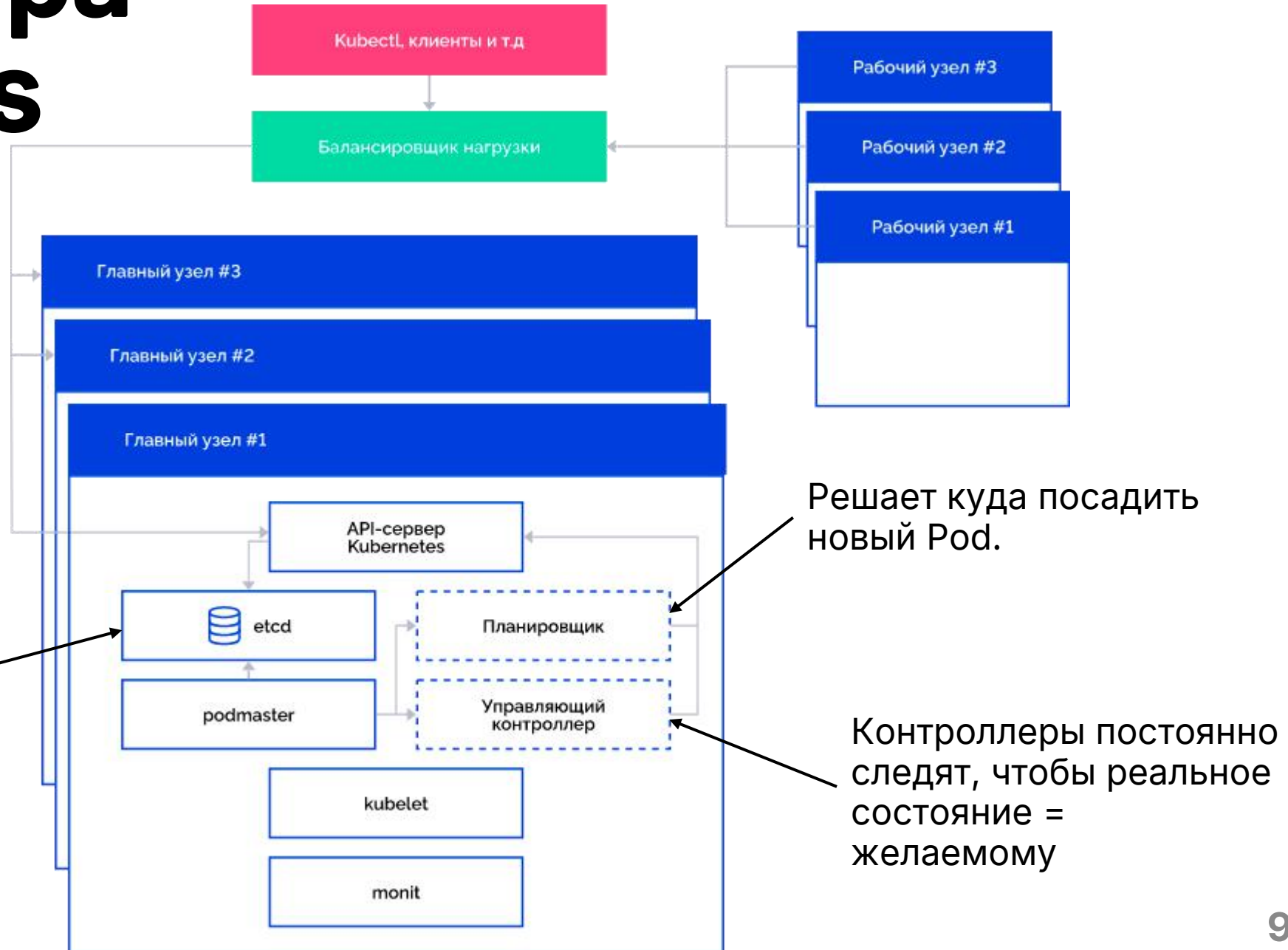
- выделять ресурсы (CPU, память)
- перезапускать при сбое
- обеспечивать сетевую связность между ними
- автоматически масштабировать под нагрузкой
- следить за логами и метриками

Вручную это делать тяжело.

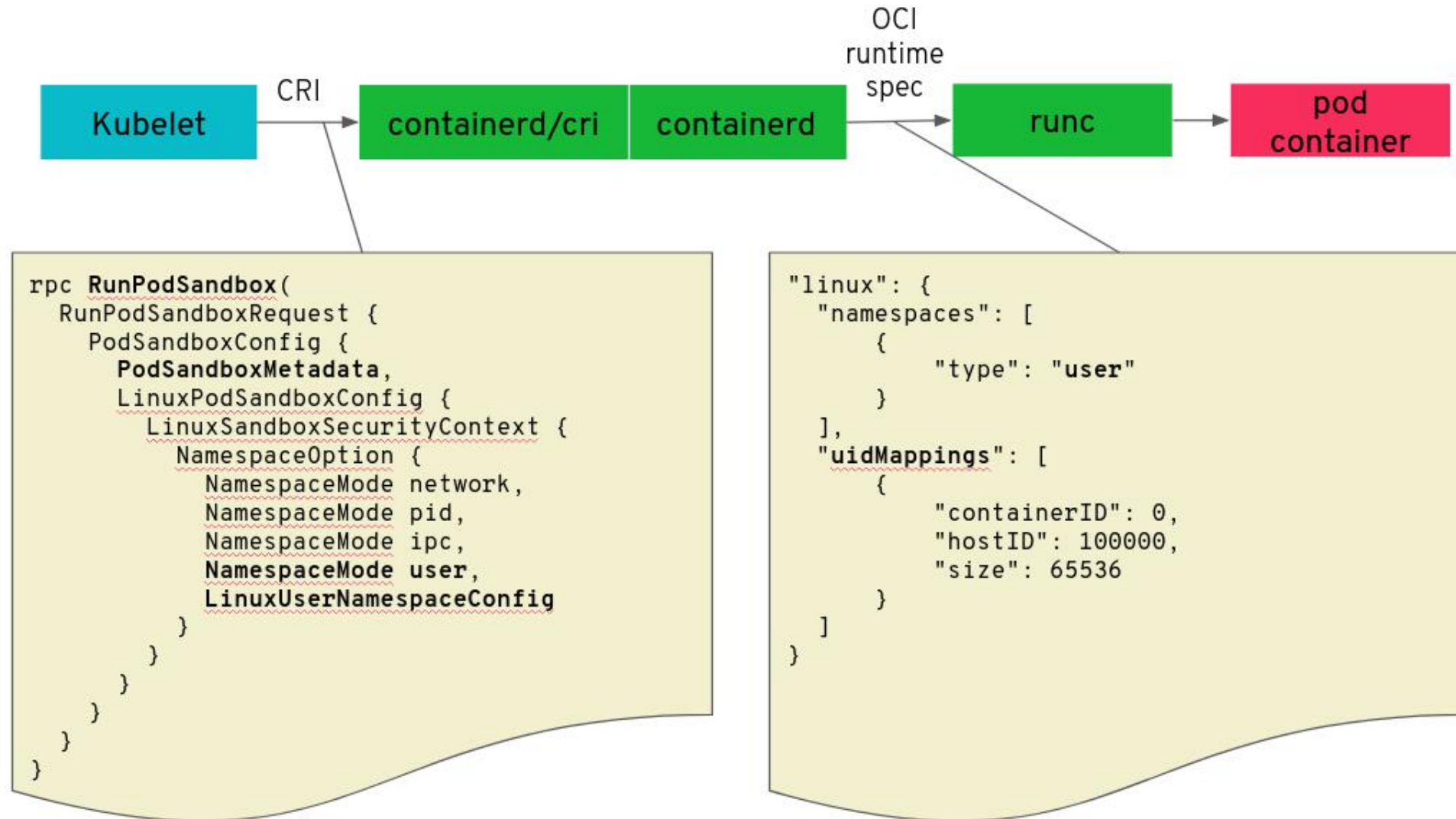
Архитектура Kubernetes

В сам Kubernetes загружается схема о том, как и что должно работать. Он **отслеживает** функционирование кластера и **сопоставляет** его с желаемым, а если возникает потребность, то **заново запускает** необходимые компоненты.

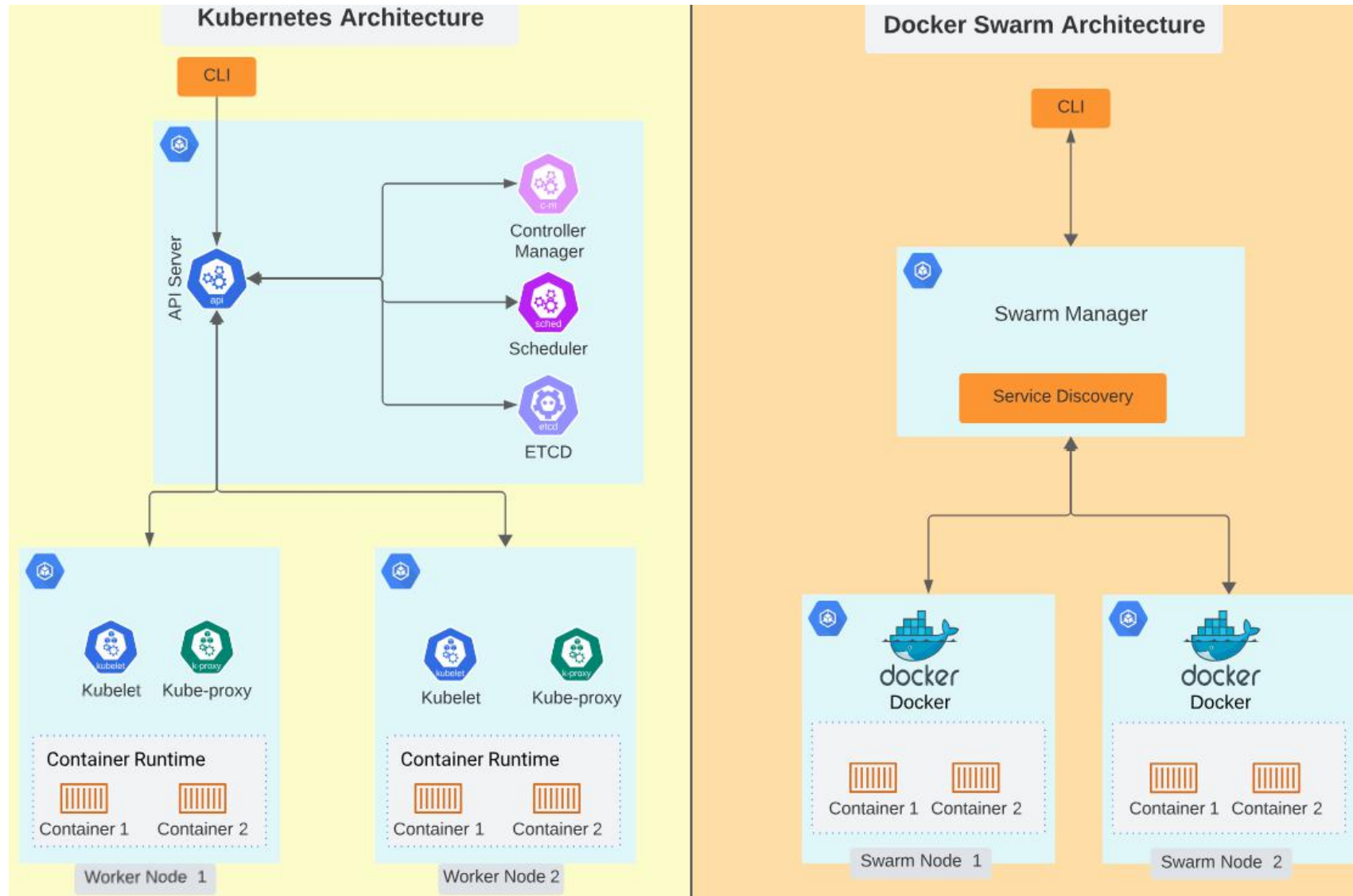
хранит всё состояние кластера



Взаимодействие Kubernetes с containerd через CRI, конфигурация



Почему просто не использовать Docker swarm?



Нетворкинг и порты

Нужно, если несколько сервисов/портов (nginx на 80, api на 8080, ...).

В docker-compose мы "маппим" порты контейнера на порты хоста напрямую:

```
1 services:
2   web:
3     image: nginx
4     ports:
5       - "8080:80"
6       - "127.0.0.1:9000:9000"
7       - "3306"
```

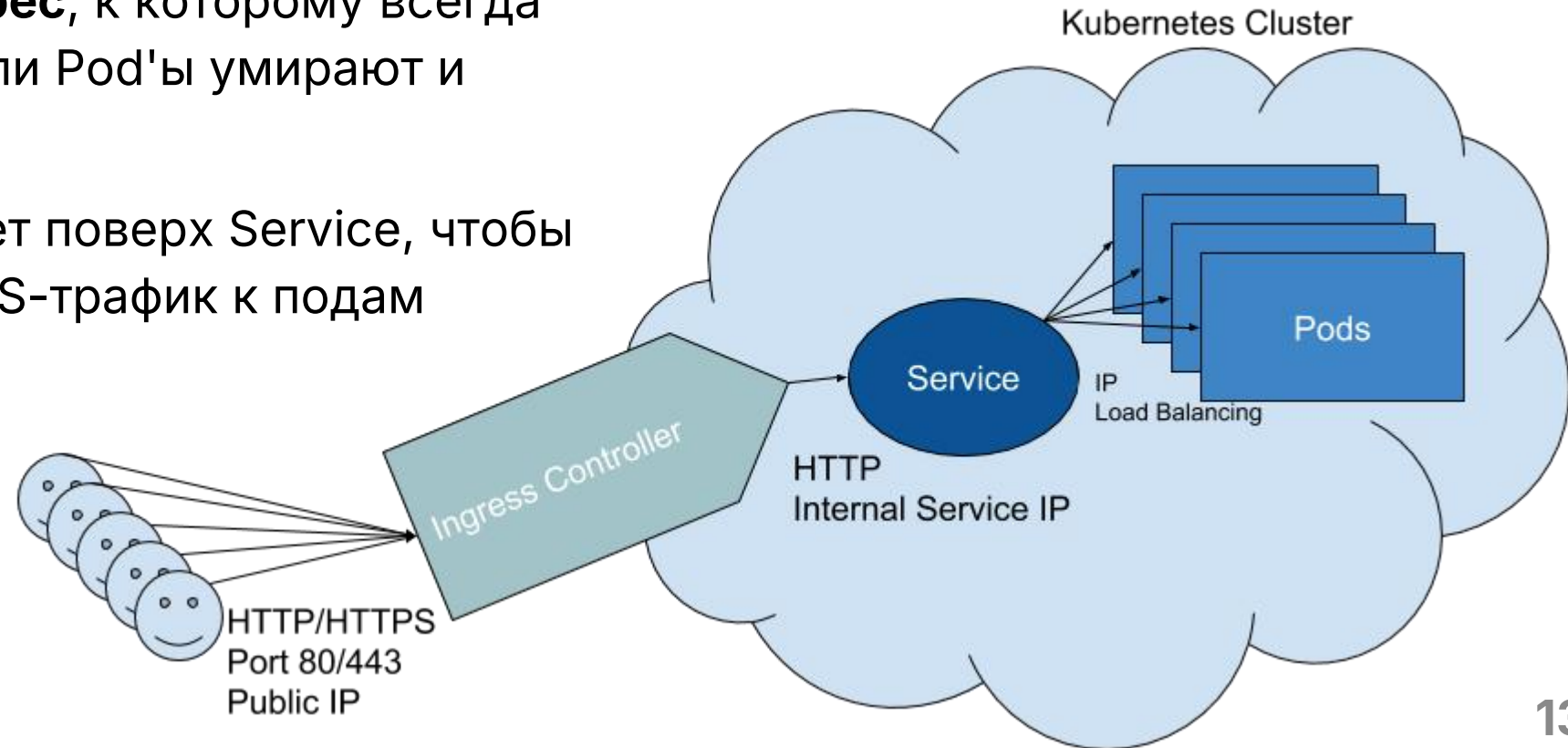
В K8s порты не надо мапить на хост вручную, а создаётся Service (или Ingress). Он абстрагирует доступ к подам, балансирует трафик и решает, как сделать приложение видимым снаружи.

Service, Ingress

Service — это стабильный IP + DNS для группы Pod'ов. Без Service каждый Pod имеет свой изменчивый IP, который меняется при перезапуске или масштабировании.

Service даёт **постоянный адрес**, к которому всегда можно обращаться, даже если Pod'ы умирают и рождаются заново.

Ingress контроллер - работает поверх Service, чтобы пустить внешний HTTP/HTTPS-трафик к подам



Apptainer (бывший Singularity) — контейнеры для HPC

Особенности:

- Разработан для HPC-кластеров;
- Не требует постоянного демона (в отличие от Docker). Фоновые контейнеры запускаются без демона — просто отдельный процесс;
- Контейнеры запускаются от имени обычного пользователя;
- Формат образов SIF (Singularity Image Format) — один неизменяемый файл;
- Сохраняет полную совместимость с Docker-образами (**apptainer pull docker://...**)



Apptainer

- **Без root-привилегий.** Fake root через user namespaces. Процесс внутри контейнера видит uid=0, но на хосте это обычный пользователь. Это безопасно в многопользовательском окружении.
- **Слои почти не используются,** каждый билд - почти полностью новый образ
- Не так развит нетворкинг

ОСНОВНЫЕ КОМАНДЫ

Скачать Docker-образ и конвертировать в .sif

```
apptainer pull tensorflow.sif docker://tensorflow/tensorflow:latest-gpu
```

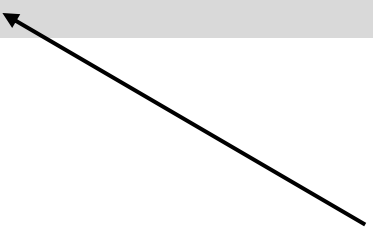
Запустить (аналог docker run)

```
apptainer run tensorflow.sif  
apptainer exec tensorflow.sif python script.py  
apptainer shell tensorflow.sif
```

запускает %runscript
из дефайна



оболочка внутри контейнера
(интерактивный режим)



.def-файл (аналог Dockerfile)

Берём готовый
Docker-образ с
Docker Hub (или
другого registry),
Ubuntu 24.04 LTS

Команды во время
сборки контейнера
(= RUN)

Переменные
окружения
(= ENV)

```
1 Bootstrap: docker
2 From: ubuntu:24.04
3 %post
4     apt-get update && apt-get install -y python3 python3-pip
5     pip3 install numpy pandas torch
6 %environment
7     export PATH=/usr/local/bin:$PATH
8 %runscript
9     python3 /app/main.py
```

Сборка

Сборка своего контейнера
(из definition-файла .def)

```
sudo apptainer build myapp.sif myapp.def
```

Podman

Альтернатива Docker, запускает контейнеры без демона

- Также как и Arptainer запускает контейнеры от имени обычного пользователя;
- Совместим с Docker-образами и командами;
- Поддерживает поды, как в Kubernetes.



podman